L2B-1. Stored Procedures and Functions

Key problem

When should reusable database operations be stored inside the DBMS rather than repeated in application code?

Motivating Problem

A repeated operation should not be copied into every application

A video-rental application repeatedly needs to answer:

How many films are available for a given category/language/store/month?

Without a stored routine:

Each application rewrites similar SQL logic.

Bug fixes and rule changes must be propagated to all clients.

With a stored routine:

The DBMS exposes a named operation.

Applications call the routine with parameters.

The database administrator can inspect, authorize, and version the routine.

This is not only programming convenience; it is database-interface design.

Part I — Stored Routine Concepts

Stored Routine

A database object containing SQL logic that can be invoked later

- A stored routine is stored in the DBMS catalog.

  The definition belongs to a database schema and can be inspected administratively.

- The body can contain SQL statements and procedural constructs.

  Examples: SELECT, INSERT, variable assignment, IF/CASE, loops.

- The routine is invoked by name.

  A procedure is invoked with CALL.

  A function is invoked as an expression and returns a value.

- Stored routines are useful only when the database is the right place for the logic.

Why Store Logic in the DBMS?

Benefits and costs must be evaluated together

Potential benefit	Explanation
	Multiple applications can call the same database operation.
Centralized rule	Changing routine logic changes the operation at the DBMS boundary.
Reduced network round trips	A multi-step operation can be invoked by a single CALL.
Controlled access	Applications may be granted EXECUTE on a routine instead of broad table privileges.
Maintenance risk	Logic may become hidden, DBMS-specific, and harder to test/debug.

Database Programmability Objects

Choose the right object for the job

Object	Invocation	Main purpose
View	SELECT from a relation name	Reusable logical relation / abstraction
Procedure	CALL procedure_name(...)	Multi-step database operation
Function	Use inside an expression	Reusable scalar computation
Trigger	Automatic on INSERT/UPDATE/DELETE	Reaction to a data-change event

View = interface → **Procedure = operation** → **Function = value** → **Trigger = reaction**

Procedure vs Function

Same family, different role

Aspect	Stored procedure	Stored function
Invocation	CALL proc(...)	Used as an expression, e.g., SELECT f(...)
Return style	Result set and/or OUT parameters	A single return value
Typical use	Operation or workflow	Reusable computation
Example	Find films by parameters	Compute rental count for store/month/year
Design question	Does this action change or orchestrate work?	Does this compute a value for a query?

Function Example

```
create function dept_count(dept_name varchar(20))
    returns integer
    begin
    declare d_count integer;
        select count( *) into d_count
        from instructor
        where instructor.dept_name= dept_name
    return d_count;
    end
```

Figure 5.6 Function defined in SQL.

What the figure shows:

A function named **dept_count**.

An **input parameter** for a department name.

A query that counts instructors in that department.

A **RETURN statement** that returns the count.

Why it matters:

The logic can be reused in later SQL queries.

The function is a database object, not application-only code.

Step-by-Step: dept_count

```
CREATE FUNCTION
dept_count(dept_name varchar(20))
RETURNS integer
BEGIN
    DECLARE d_count integer;
    SELECT COUNT(*) INTO d_count
    FROM instructor
    WHERE instructor.dept_name =
dept_name;
    RETURN d_count;
END
```

Step	Meaning
1. Signature	Function name and input parameter.
2. RETURNS	The function returns one integer value.
3. Local variable	d_count stores the intermediate result.
4. SELECT INTO	Query result is assigned to a variable.
5. RETURN	The scalar value is returned to the caller.

Using A Function In A Query

A function can appear where a scalar expression is allowed

Query using dept_count

```
SELECT dept_name, budget
FROM department
WHERE dept_count(dept_name) > 12;
```

- The function is evaluated for department names used by the query.

  Each call returns a scalar count.
- This is why functions fit expression-level computation.

  They can be composed inside SELECT, WHERE, or other expressions depending on DBMS rules.

Design caution:

- A function used inside a query may be called many times; this can affect performance.
- Do not hide expensive repeated work without measurement.

Source: DSC 7e, Ch.5 §5.2.

Table Function Concept

```
create function instructor_of (dept_name varchar(20))
        returns table (
                ID varchar (5),
                name varchar (20),
                dept_name varchar (20),
                salary numeric (8,2))
return table
        (select ID, name, dept_name, salary
        from instructor
        where instructor.dept_name = instructor_of.dept_name);
```

Figure 5.7 Table function in SQL.

The SQL standard supports table functions conceptually.

A table function returns a relation rather than a scalar value.

MySQL focus:

We will implement scalar stored functions and procedures with result sets.

Routine Metadata In MySQL

Stored routines are inspectable database objects

```
SHOW PROCEDURE STATUS
WHERE Db = 'sakila';

SHOW FUNCTION STATUS
WHERE Db = 'sakila';

SELECT routine_schema, routine_name,
routine_type,
       data_type, security_type,
sql_data_access
FROM information_schema.routines
WHERE routine_schema = 'sakila';
```

Administration perspective:

Do not only test that a routine runs.

Inspect how the DBMS records it.

Useful metadata:

routine type: PROCEDURE or FUNCTION.

security type: DEFINER or INVOKER.

SQL data access characteristic.

Part II — Stored Procedures In MySQL

Writing A MySQL Routine Script: Six Steps

Read the whole script as a sequence of client and server actions

```
DELIMITER $$
CREATE PROCEDURE count_films(
    IN  p_language_id INT,
    OUT p_count INT
)
BEGIN
    SELECT COUNT(*) INTO p_count
    FROM film
    WHERE language_id =
p_language_id;
END $$
DELIMITER ;
CALL count_films(1, @n);
SELECT @n;
```

Step	Meaning
1. DELIMITER $$	Client waits for $$ before sending statement.
2. CREATE PROCEDURE	Server creates a stored routine object.
3. BEGIN ... END	Routine body contains SQL statements ending with ;
4. END $$	Client sends the whole CREATE statement.
5. DELIMITER ;	Restore normal statement terminator.
6. CALL / SELECT	Run procedure and inspect OUT parameter.

Source: MySQL 8.4 Reference Manual, §27.1 and §15.1.17

Common DELIMITER Mistakes

Debug the script before debugging the database logic

Mistake	Symptom	Correction
No DELIMITER before CREATE	Syntax error near first internal ;	Change delimiter before routine definition.
Missing END $$	Client waits or sends partial statement	Close body with END and current delimiter.
No DELIMITER ; after CREATE	Later SQL commands behave unexpectedly	Restore normal delimiter.
Expect DELIMITER in SHOW CREATE	SHOW CREATE does not show it	Remember: DELIMITER is a client command.

- First confirm the routine object exists: SHOW CREATE PROCEDURE.
- Then test behavior with several CALL statements.
- Separate script-delimiter errors from routine-logic errors.

CREATE PROCEDURE Skeleton

Minimal MySQL form used in the lab

```
DELIMITER $$

CREATE PROCEDURE procedure_name(
    IN  p_input  INT,
    OUT p_output INT
)
BEGIN
    -- SQL statements and control flow
END $$

DELIMITER ;
```

DELIMITER is a client-side command.

It lets the client send the whole routine body to the server.

CREATE PROCEDURE creates a stored routine in the current schema.

The parameter list defines how callers pass values in and out.

BEGIN ... END defines a compound statement.

Declarations and executable statements appear inside the block.

Procedure Parameters

IN, OUT, and INOUT

Mode	Caller supplies value?	Routine can change value?	Typical use
IN	Yes	Local only	Filter condition, identifier, option
OUT	No meaningful input	Yes, returned to caller	Return a count/status/output value
INOUT	Yes	Yes, returned to caller	Update a running value or state variable

Design rule: use OUT parameters for values the procedure computes for the caller; use result sets when the natural output is a table-like query result.

Example: count_films

A first procedure with IN and OUT parameters

```
DELIMITER $$
CREATE PROCEDURE count_films(
    IN  p_language_id INT,
    OUT p_count INT
)
BEGIN
    SELECT COUNT(*)
    INTO p_count
    FROM film
    WHERE language_id = p_language_id;
END $$
DELIMITER ;
```

Step 1: caller passes p_language_id.

Step 2: procedure runs one SELECT query.

Step 3: SELECT … INTO assigns the count to p_count.

Step 4: OUT parameter becomes available to the caller after CALL.

Why this is a procedure:

The caller invokes a named database operation with CALL.

Running count_films

Observe the procedure output

```
CALL count_films(1, @n);
SELECT @n AS number_of_films;

SHOW CREATE PROCEDURE count_films;
```

What to observe:

- @n is a user variable in the current session.
- The procedure assigns a value to @n through the OUT parameter.
- SHOW CREATE PROCEDURE confirms the stored definition.

Mini-check:

- What changes if p_language_id has no matching rows?
- What output form would you use if the routine returned many film rows?

Inspecting, Replacing, and Dropping Routines

Stored routines are database objects that must be managed

```
-- Inspect a procedure or function
SHOW CREATE PROCEDURE count_films;
SHOW CREATE FUNCTION
rental_count_by_store;

-- Metadata
SELECT routine_name, routine_type,
data_access
FROM information_schema.routines
WHERE routine_schema = 'sakila';

-- Safe redefinition pattern
DROP PROCEDURE IF EXISTS count_films;
-- then CREATE PROCEDURE again
```

- **SHOW CREATE** confirms the stored definition, not just that the script ran.
- **INFORMATION_SCHEMA.ROUTINES** helps compare procedures and functions.
- For lab iteration, **DROP IF EXISTS + CREATE** is clearer than editing in place.
- Document routine inputs, outputs, data access type, and test calls in your report.

Procedure Output Forms

A procedure may produce output in more than one way

Output form	How it appears	Best for
Result set	Procedure executes a SELECT statement	Returning table-like results
OUT parameter	Caller passes a variable and reads it after CALL	Returning small scalar results/status
Side effects	Procedure modifies tables	Multi-step database operation
Messages/errors	Procedure signals or raises conditions	Validation and controlled failure

Design question: is the routine mainly producing a report, computing a value, or performing an operation?

UET Example: film_in_stock

A procedure that returns inventory rows and a count

```
CREATE PROCEDURE film_in_stock(
    IN p_film_id INT,
    IN p_store_id INT,
    OUT p_film_count INT
)
READS SQL DATA
BEGIN
    SELECT inventory_id
    FROM inventory
    WHERE film_id = p_film_id
      AND store_id = p_store_id
      AND inventory_in_stock(inventory_id);

    SELECT FOUND_ROWS() INTO p_film_count;
END
```

Purpose:

Find available inventory IDs for a film in a store.

Return the number of matching rows through an OUT parameter.

Important dependency:

The procedure calls inventory_in_stock(...), a stored function.

Step-by-Step: film_in_stock

How the procedure executes

Step	Operation	Question to ask
1	Receive film_id and store_id	What operation is being parameterized?
2	Scan/filter inventory	Which rows are candidates?
3	Call inventory_in_stock	Is this inventory item currently available?
4	Return inventory_id rows	Does the caller expect a result set?
5	Assign p_film_count	How does caller retrieve the count?

Analysis habit: before writing a routine, write the inputs, output form, tables touched, and dependency on other routines.

Lab Requirement: displayFilmInfo

Translate a requirement into branch logic

Create a stored procedure displayFilmInfo(category_id, language_id).

If both parameters are specified:

Return films matching both category and language.

If language_id = 0:

Return films matching category_id only.

If category_id = 0:

Return films matching language_id only.

If both are 0:

Define a reasonable behavior explicitly before coding.

Deliverable: procedure definition, test calls, and explanation of each branch.

Decision Table Before SQL

Make the branching explicit first

category_id	language_id	Filtering logic
> 0	> 0	category_id AND language_id
> 0	= 0	category_id only
= 0	> 0	language_id only
= 0	= 0	Either reject or return all films; state your choice

This table is part of the design. Code should implement a stated decision, not hide the decision.

displayFilmInfo — Skeleton

Start with parameters and the common join

```
DELIMITER $$
CREATE PROCEDURE displayFilmInfo(
  IN p_category_id INT,
  IN p_language_id INT
)
BEGIN
  -- Branches will decide which WHERE predicates to use.
  -- Common tables:
  -- film, film_category, category, language
END $$
DELIMITER ;
```

Separate the stable part from the variable part.

Stable: tables and joins needed to describe films.

Variable: which filters are applied.

This reduces copy-paste mistakes.

In production, dynamic SQL or optional predicates may be considered, but this lab focuses on readable branching.

displayFilmInfo — Branching Pattern

Use IF / ELSEIF to match the decision table

```
IF p_category_id > 0 AND p_language_id > 0
THEN
    -- SELECT ... WHERE category_id = p_category_id
    --            AND language_id = p_language_id;
ELSEIF p_category_id > 0 AND p_language_id = 0
THEN
    -- SELECT ... WHERE category_id =
p_category_id;
ELSEIF p_category_id = 0 AND p_language_id > 0
THEN
    -- SELECT ... WHERE language_id =
p_language_id;
ELSE
    -- reject or return all films; document the
choice
END IF;
```

Each branch should be testable.

Prepare one CALL for each row in the decision table.

Avoid ambiguous behavior.

If both parameters are 0, your procedure should not silently surprise users.

Lab report:

Include the decision table and test output for each branch.

Testing displayFilmInfo

A procedure is not finished until all branches are tested

```
CALL displayFilmInfo(1, 1);
-- both filters
CALL displayFilmInfo(1, 0);
-- category only
CALL displayFilmInfo(0, 1);
-- language only
CALL displayFilmInfo(0, 0);
-- boundary case

SHOW CREATE PROCEDURE
displayFilmInfo;
```

For each test call, record:

- Which branch should execute?
- How many rows are returned?
- Does the result match the intended filter?

Evidence:

- Procedure definition.
- Four test calls and outputs.
- Short explanation of the boundary case.

In-Class Exercise 1

Trace the branch before running the code

Assume displayFilmInfo implements the decision table exactly.

For each call, write the expected WHERE condition:

CALL displayFilmInfo(2, 1);

CALL displayFilmInfo(2, 0);

CALL displayFilmInfo(0, 3);

CALL displayFilmInfo(0, 0);

Then compare with the actual output after running the procedure.

Purpose: learn to reason about stored routine behavior without treating it as a black box.

Part III — Procedural SQL In MySQL

Compound Statement Structure

Declarations appear before executable statements

```
BEGIN
    DECLARE local variables;
    DECLARE conditions;
    DECLARE cursors;
    DECLARE handlers;

    -- executable SQL statements
END
```

- MySQL stored programs use compound statements to group logic.
- Declarations must appear before executable statements.
- The order of declaration types matters in MySQL documentation.
- This is why many cursor examples begin with several DECLARE statements before OPEN.

Variables and SELECT ... INTO

Store intermediate results inside a routine

```
DECLARE v_count INT b 0;

SELECT COUNT(*)
INTO v_count
FROM customer
WHERE active = 1;

SELECT v_count AS active_customers;
```

Local variables exist only inside the routine invocation.

SELECT ... INTO assigns a query result to variables.

Use variables for:

- intermediate counts;
- status flags;
- values needed by later conditions.

If the SELECT returns unexpected rows, the routine may fail or behave incorrectly; test carefully.

IF / ELSEIF / ELSE

Branching inside stored programs

```
IF condition THEN
    statement_list;
ELSEIF another_condition THEN
    statement_list;
ELSE
    statement_list;
END IF;
```

Use IF when different SQL statements should run under different conditions.

Design advice:

- Write the decision table first.
- Create one test case per branch.
- Handle boundary cases explicitly.

CASE Statement

Useful when choices are based on discrete values

```
CASE status_code
    WHEN 1 THEN
        SET v_status = 'NEW';
    WHEN 2 THEN
        SET v_status = 'PAID';
    ELSE
        SET v_status = 'UNKNOWN';
END CASE;
```

CASE can make multi-way choices clearer than a long IF chain.

Best use:

- mapping codes to behaviors;
- choosing among known categories;
- handling business-rule cases with a small number of options.

Do not use procedural CASE when a declarative SQL expression is simpler.

Loops In Stored Programs

Use only when set-oriented SQL is not enough

Loop form	Typical meaning	Caution
WHILE	Repeat while condition is true	Condition must eventually become false
REPEAT	Run at least once until condition becomes true	Check exit condition carefully
LOOP + LEAVE	General loop with explicit exit	More flexible but easier to misuse

DBMS principle: prefer set-oriented SQL when possible. Use loops when the operation genuinely requires procedural row-by-row logic.

When Do We Need A Cursor?

A cursor iterates over rows returned by a query

A cursor is used to process query-result rows one at a time.

Typical pattern:

SELECT produces a result set;

FETCH reads one row into variables;

routine processes the current row;

loop repeats until no more rows.

Use cases:

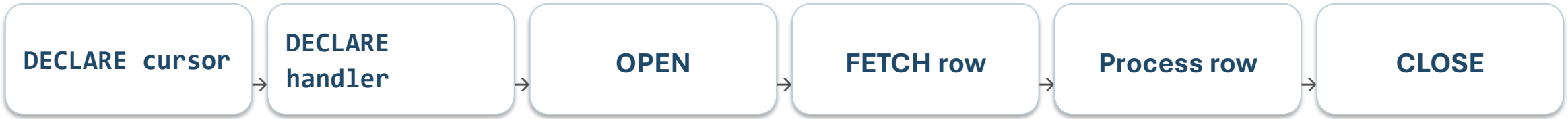
procedural update that cannot be expressed cleanly as one set-based SQL statement;

educational examples for control flow.

Caution: cursor-based row-at-a-time processing can be slower and harder to maintain.

Cursor Lifecycle

The minimum sequence students must understand



- DECLARE CURSOR binds a SELECT statement to a cursor name.
- DECLARE HANDLER defines what happens when FETCH reaches the end.
- OPEN executes the cursor query and prepares the result set.
- FETCH transfers current row values into variables.
- CLOSE releases cursor resources.
- Most bugs come from missing handler logic or incorrect loop exit conditions.

Cursor Skeleton

Read it before adapting it

```
DECLARE done BOOL DEFAULT FALSE;
DECLARE v_film_id INT;
DECLARE cur CURSOR FOR
  SELECT film_id FROM film ORDER BY film_id LIMIT 10;
DECLARE CONTINUE HANDLER FOR NOT FOUND
  SET done = TRUE;

OPEN cur;
read_loop: LOOP
  FETCH cur INTO v_film_id;
  IF done THEN
    LEAVE read_loop;
  END IF;
  -- process v_film_id here
END LOOP;
CLOSE cur;
```

Step-by-step:

done is initially FALSE.

When FETCH has no row, the handler sets done = TRUE.

The loop exits after detecting done.

Important:

Always close the cursor.

Keep cursor logic small and well-tested.

Lab Extension: Discount Low-Rental Films

Why cursor logic appears in the existing lab

Task idea:

Reduce rental price by p% for n films with the lowest rental counts in the previous month.

Why it is harder than the earlier tasks:

Need to find a set of film_id values based on aggregate statistics.

Then update selected films one by one or by a set-based statement.

Project lesson:

A procedural implementation is not automatically the best implementation.

Part IV — Stored Functions In MySQL

CREATE FUNCTION Skeleton

A stored function returns one value

```
DELIMITER $$
CREATE FUNCTION
function_name(p_input INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_result INT DEFAULT
0;
    -- compute v_result
    RETURN v_result;
END $$
DELIMITER ;
```

- A function must return a value.

  The return type appears after RETURNS.
- A function can be used inside SQL expressions.

  For example: SELECT function_name(...).
- MySQL requires routine characteristics such as data-access information in many deployments.

  Examples: READS SQL DATA, DETERMINISTIC.

Sakila Example: inventory_in_stock

A function that answers whether one inventory item is available

```
CREATE FUNCTION
inventory_in_stock(p_inventory_id INT)
RETURNS BOOLEAN
READS SQL DATA
BEGIN
    DECLARE v_rentals INT;
    DECLARE v_out INT;

    SELECT COUNT(*) INTO v_rentals
    FROM rental
    WHERE inventory_id = p_inventory_id;

    IF v_rentals = 0 THEN
        RETURN TRUE;
    END IF;
    -- continue: check unreturned rentals
END
```

Question answered:

Is this specific inventory item currently in stock?

First step:

Count rental records for that inventory item.

If never rented:

The item is considered available.

Return value:

TRUE or FALSE, usable in a WHERE condition.

Step-by-Step: inventory_in_stock

Second check: is there an unreturned rental?

```
SELECT COUNT(rental_id) INTO v_out
FROM inventory
LEFT JOIN rental USING(inventory_id)
WHERE inventory.inventory_id =
p_inventory_id
  AND rental.return_date IS NULL;

IF v_out > 0 THEN
    RETURN FALSE;
ELSE
    RETURN TRUE;
END IF;
```

If there is a rental row with return_date IS NULL:

The item is currently out.

The function returns FALSE.

Otherwise:

The item is available.

The function returns TRUE.

This function becomes reusable inside procedures such as film_in_stock.

Using inventory_in_stock

A function can filter query results

```
SELECT inventory_id
FROM inventory
WHERE film_id = 1
  AND store_id = 1
  AND
inventory_in_stock(inventory_id);

SELECT inventory_in_stock(1) AS
is_available;
```

The function is used as a Boolean predicate.

- Only inventory IDs for which the function returns TRUE are returned.

This is convenient but can be expensive.

- The function may run once per candidate row.

Analysis question:

- Could the same result be expressed as one SQL query without a function?

Function Task

Compute monthly rentals for a store

Write a function that returns the total number of film rentals for a store in a given month and year.

Inputs:

store_id

month

year

Return:

an integer rental count.

Then use the function in a SELECT query to show stores and their rental counts for February 2006.

Reasoning before code:

Which tables contain store and rental information?

Which date column should be filtered?

Rental Count Function — Design Skeleton

Write the table join and date filter before coding the function

```
CREATE FUNCTION rental_count_by_store_month(
p_store_id INT, p_month INT,  p_year INT )
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;

    SELECT COUNT(*) INTO v_count
    FROM rental AS r
    JOIN inventory AS i ON r.inventory_id =
i.inventory_id
    WHERE i.store_id = p_store_id
      AND MONTH(r.rental_date) = p_month
      AND YEAR(r.rental_date) = p_year;

    RETURN v_count;
END
```

Step-by-step:

Join rental to inventory to identify the store.

Filter by month/year on rental_date.

Count matching rows.

Return the count.

Implementation caution:

Functions in WHERE/SELECT can be called repeatedly; measure for large data.

Procedure vs Function After Examples

Now compare based on behavior you have seen

Question	Choose procedure when...	Choose function when...
How is it invoked?	Caller should use CALL.	Caller should use it as an expression.
What does it produce?	A result set, side effect, or OUT values.	One scalar return value.
What is the mental model?	Run an operation.	Compute a value.
Example from lab	displayFilmInfo.	rental_count_by_store_month.
Project warning	Too much workflow in DB can be hard to test.	Repeated function calls may be costly.

Homework B1 — Required Tasks

Task	Requirement	Evidence
HW1	Modify count_films to count films by category_id and language_id.	CREATE script, CALL tests, expected vs actual counts.
HW2	Complete displayFilmInfo with all four parameter cases.	Decision table, procedure code, 4 test calls.
HW3	Write a function rental_count(store_id, month, year).	Function code, SELECT using the function for all stores.
HW4	Explain one case where procedure is better than function.	Short paragraph using your implemented examples.

Submission: one .sql file + screenshots/output + short explanation for each task.

Homework B1 — Extension Options

HOMEWORK

Choose one extension if your project needs deeper procedural logic

- Extension A — set-based alternative:
  - Rewrite the “discount low-rental films” requirement using one UPDATE with a subquery instead of a cursor.
  - Explain which version is easier to test and which may be more efficient.
- Extension B — cursor version:
  - Use a cursor to fetch n low-rental film_id values and update them one by one.
  - Show cursor lifecycle: DECLARE → OPEN → FETCH loop → CLOSE.
- Extension C — error handling:
  - Add a handler for NOT FOUND or invalid input and explain what it catches.

Project link: use an extension only if row-by-row logic, error handling, or a stored operation is genuinely needed.

Source: UET Lab 03; DSC 7e Ch.5 §5.2

Beyond the Core

Explore how other DBMSs implement procedural database logic

DBMS	Topic	Official reference
PostgreSQL	PL/pgSQL functions and procedures	PostgreSQL docs: PL/pgSQL
Oracle	PL/SQL procedures, functions, and packages	Oracle Database PL/SQL Language Reference
SQL Server	T-SQL stored procedures and scalar functions	Microsoft Learn: Transact-SQL procedures/functions
MySQL	Event Scheduler and stored-object access control	MySQL 8.4 Manual §27 Stored Objects

Beyond MySQL: PostgreSQL PL/pgSQL

Same idea, different procedural language and routine model

- PostgreSQL supports user-defined functions written in SQL, PL/pgSQL, C, and other procedural languages.
- PL/pgSQL functions can return scalar values, rows, sets, or void.
- CREATE OR REPLACE FUNCTION is commonly used to update definitions without dropping dependent objects.
- Security-definer functions require careful search_path design.
- Project exploration:
  - Implement rental_count in PostgreSQL and compare function syntax, return types, and metadata with MySQL.

Official docs: https://www.postgresql.org/docs/18/xplang.html | https://www.postgresql.org/docs/18/plpgsql-overview.html

Beyond MySQL: Oracle PL/SQL Packages

Enterprise systems often group routines into packages

- Oracle PL/SQL supports procedures, functions, triggers, exceptions, cursors, and packages.
- A package groups related procedures, functions, types, variables, cursors, and exceptions.
- Package specification is the public interface; package body contains implementation details.
- Changing the body can preserve the public API and reduce dependencies on implementation details.
- Project exploration:
  - Design a package specification for rental operations: availability check, monthly count, and overdue-list procedure.

Official docs: https://docs.oracle.com/en/database/ | PL/SQL Language Reference / Packages

Source: Oracle Database PL/SQL Language Reference; Oracle package documentation

Beyond MySQL: SQL Server T-SQL Routines

Transact-SQL has its own procedure/function conventions

- SQL Server stored procedures are created with CREATE PROCEDURE or CREATE OR ALTER PROCEDURE.
- T-SQL procedures can accept input parameters and return output parameters or status values.
- SQL Server supports scalar-valued and table-valued user-defined functions.
- Stored procedures, functions, and triggers have different restrictions and invocation contexts.
- Project exploration:
  - Port displayFilmInfo to T-SQL and identify which parts are syntactically different from MySQL.

Official docs: https://learn.microsoft.com/sql/t-sql/statements/create-procedure-transact-sql

Source: Microsoft Learn, CREATE PROCEDURE and CREATE FUNCTION (Transact-SQL)